ReactJS Technical Assignment: Quiz Platform (Frontend)

Overview

Build the frontend of a Quiz Platform in ReactJS: quiz list, quiz admin, an in-app quiz-taking flow, results and a leaderboard. There is no backend and no API, real or mocked. All data lives in the browser: it is seeded from a local JSON file and persisted in localStorage, as described in the next section.

We use this assignment to assess how you structure a React application, manage state across a multi-step flow, handle timing and persistence on the client, and build a polished, responsive, accessible UI. We value clean, readable, well-tested code over the number of features.

If you run short of time, finish the core flow well and list what you skipped in the README.

Tech stack and data

Use React 18+ with TypeScript, built with Vite. Routing, styling and state-management libraries are your choice; justify them briefly in the README. Do not use a ready-made quiz component or quiz template.

The app runs fully in the browser with no server. Use the provided `seed.json` (5 users, 3 countries, 5 quizzes and some past attempts, including one in progress) and include it in your repo; you may add to it. On first load, copy it into localStorage (or IndexedDB); after that, every change is saved there so data survives page reloads. Provide a "Reset data" action that restores the seed.

Keep all data handling in a separate layer of your choice (for example a service, repository or store), so components never read or write browser storage directly. This layer should support at least the following operations:

Function	Returns / does
`getQuizzes(userId, country?)`	Quizzes filtered by country, each with `completed` and `bestScore` for the user
`getQuiz(quizId)`	Title, description, time limit, retakes allowed, question count

Function	Returns / does
`createQuiz(data)` / `updateQuiz(id, data)` / `deleteQuiz(id)`	Admin changes, including assigned countries
`startOrResumeAttempt(quizId, userId)`	A new attempt, or the user's in-progress one
`getCurrentQuestion(attemptId)`	Current question without correct answers, index, total and remaining time
`submitAnswer(attemptId, questionId, optionIds)`	Records the answer and returns the next question
`completeAttempt(attemptId)`	Score, percentage, correct/incorrect counts and time taken
`getLeaderboard(quizId, limit)`	Rank, user name, score and completion time

Scoring and timing are computed inside the data service from stored timestamps, never from component state, and the UI must never read correct answers before an answer is submitted. Add an optional simulated delay (for example 300 to 800 ms) so loading states can be demonstrated.

Screens and features

1. Quiz list

- A user switcher (simple dropdown, no real login) and a country filter.
- Quizzes the user has completed are clearly marked with their prior score; if retakes are not allowed, opening one shows the prior result instead of the Start screen.
- Loading skeletons and an empty state.

2. Quiz admin

- List page with search, sort and pagination (client-side is fine), plus edit and delete with a confirmation dialog.
- Add and Edit forms sharing one component: title, description, time limit (overall and/or per question), retakes allowed, and assigned countries (multi-select).
- A dynamic question builder: add, remove and reorder questions; choose single-choice or multiple-choice; add and remove options; mark the correct option(s).
- Validation with clear inline errors: title required, at least one question, at least two options per question, at least one correct option, and exactly one for single-choice.

- Warn before leaving a form with unsaved changes.

3. Quiz attend

- **Start screen:** title, description, number of questions, time limit and a Start Quiz button.
- **One question at a time** (not a long scrollable form) with a progress bar and a "Question X of N" label.
- **Question types:** single-choice (radio behaviour) and multiple-choice (checkbox behaviour); image-based options if time permits.
- **Timer:** a visible countdown, overall and/or per question, calculated from the attempt's stored start time rather than a counter that resets on reload. When time runs out, auto-submit the current answer and move on (or complete the quiz). The timer must stay accurate when the tab is in the background.
- **Transitions:** smooth animation between questions, with no full page reload, and respect `prefers-reduced-motion`.
- **Resume:** if the user refreshes, closes the tab or navigates away mid-quiz, returning resumes the saved in-progress attempt at the correct question with the correct remaining time. Time that passed while away still counts.
- **Keyboard support:** options selectable with the keyboard, and Enter to submit the answer.
- **Edge cases:** no double answers when the button is clicked repeatedly, and a clear message if the quiz was edited or deleted while an attempt was in progress.

4. Results

- Final score, percentage, correct vs. incorrect counts and total time taken.
- A button to view the leaderboard for that quiz.
- A Try Again button if retakes are allowed; otherwise a friendly "already completed" state.

5. Leaderboard

- A quiz selector and a table of top users: rank, name, score and completion time (ties broken by faster time).
- Highlight the current user's row, and show their rank even if they are outside the top list.
- On small screens, adapt the table into a readable card or stacked layout.

Technical requirements

- **TypeScript:** strict mode, with typed data models and no `any` in application code.
- **Architecture:** a clear folder structure that separates UI components, pages, the data service and hooks. Put the attempt and timer logic in reusable custom hooks (for example `useQuizAttempt`, `useCountdown`), not inside page components.
- **State:** a deliberate approach (Context with useReducer, Zustand or Redux Toolkit). Components never touch localStorage directly. Screens stay in sync after changes, for example the quiz list and leaderboard reflect a newly completed attempt.
- **Routing:** deep-linkable routes for each screen, so refreshing any page works.
- **Testing:** unit tests for the data service (scoring, timing, country filtering) and the countdown hook, and component tests (React Testing Library) for the attend flow covering answering, timeout auto-submit and resume. One end-to-end test (Playwright or Cypress) of the full take-quiz path is a plus.
- **Accessibility:** semantic HTML, labelled form controls, visible focus states, focus moved to the new question on transition, and timer warnings announced to screen readers.
- **Responsive:** fully usable from 360 px mobile width to desktop, including the admin question builder.
- **Code quality:** ESLint and Prettier configured, with no lint errors and no console errors at runtime.

Bonus (optional)

- Image-based answer options, with an image upload or URL field in the admin builder.
- Dark mode that follows the system setting, with a manual toggle.
- Drag-and-drop reordering of questions in the admin builder.
- Export and import a quiz as a JSON file from the admin pages.
- Route-level code splitting, with the admin section lazy-loaded.
- Internationalisation setup with at least two languages.
- A deployed demo link (Vercel, Netlify or similar).

Submission requirements

Submit the project using one of the following methods.

Option 1: Git repository. Share a public GitHub, GitLab or Bitbucket link. We review the commit history, so commit in small, meaningful steps rather than one final commit.

Option 2: ZIP archive. Share a .zip of the complete React project, excluding `node_modules`.

Either way, include a README.md with:

1. Setup and run steps (Node version, install, dev server, build, tests).
2. How data is seeded and stored, and how to reset it.
3. A short architecture overview and the reasons for your library choices.
4. Your data models and any changes to the data-service functions listed above.
5. Assumptions, known limitations and what you would improve with more time.
6. Screenshots or a short screen recording of the attend flow on mobile and desktop (optional but appreciated).